

JasperStudio

Développement et maintenance de rapports
orientés édition par les utilisateurs métier

David THIBAU – 2026

Agenda

Jour 1 : Concepts

- Introduction à JasperStudio
- Découverte de l'interface
- Les sources de données
- Anatomie d'un rapport Jasper
- Les éléments de base

Jour 2 : Formatage de rapport

- Gestion des images et logos
- Gestion des polices
- Styles et formats
- Contrôler la pagination
- Expressions simples

Approche pédagogique

- Alternance de présentations courtes, de démonstrations et de manipulations guidées dans JasperStudio
- Un atelier pratique à la fin de chaque thème
- Aucune connaissance Java requise : les expressions sont abordées comme des formules de tableur
- Connaissance sommaire du SQL utile pour les sources de données
- Les ateliers peuvent être adaptés aux rapports de votre contexte

Introduction à JasperStudio

La suite Jasper et le rôle de JasperStudio

Cycle de vie d'un rapport

Types de fichiers

Structure du fichier .jrxml

Environnement de développement

La suite Jaspersoft

- **JasperReports Library** : le moteur Java qui génère les rapports (PDF, HTML, Excel, ...) à partir d'un modèle
 - Open source (licence LGPL), version 7.0.x
- **Jaspersoft Studio** : l'éditeur graphique de rapports, basé sur Eclipse
 - Gratuit (édition communautaire), même numéro de version que la librairie
- **JasperReports Server** : portail de publication, planification et diffusion des rapports
 - Produit commercial ; l'édition communautaire du serveur a été retirée en janvier 2024

Éditeur : Jaspersoft, successivement TIBCO, Cloud Software Group, puis **HCLSoftware** depuis juillet 2026

Les usages de JasperStudio

- **Concevoir un rapport** : mise en page, données, mise en forme
- **Modifier un rapport** existant livré par l'équipe de développement
- **Tester** : aperçu avec des données réelles, export dans tous les formats
- **Préparer la diffusion** : le rapport est ensuite exécuté par une application ou par JasperReports Server
- Ce que JasperStudio ne fait pas :
 - Envoyer ou planifier les rapports (rôle du serveur ou de l'application)
 - Modifier les données de l'entreprise (lecture seule)

Modèle de génération

La génération du rapport s'effectue en 4 étapes principales :

1. Mise au point d'un fichier **JRXML** définissant
 - La structure du rapport
 - Sa disposition et son rendu visuel
 - Les données dynamiques provenant d'une source de données (Requête, calcul, ...).
2. **Compilation** du rapport. Résultat : Fichier binaire *.jasper*
3. **Fusion** : Production du rapport avec récupération dynamique des données à partir de la source de données.
Résultat : Fichier binaire *.jrprint*
4. **Exportation** du rapport dans un format visualisable par un viewer. Le plus souvent PDF

Cycle de vie d'un rapport

- **Conception** : création du fichier *.jrxml* dans JasperStudio (assistant ou page blanche)
- **Modification** : évolution du modèle, en gardant la compatibilité avec les données et l'application
- **Test** : aperçu dans Studio avec une source de données de test, vérification de chaque format de sortie
- **Diffusion** : livraison du *.jrxml* ou du *.jasper* (et de ses ressources : images, polices, sous-rapports) à l'application ou au serveur
- À chaque génération : compilation du *.jrxml*, fusion avec les données, export dans le format demandé

Types de fichiers manipulés

- **.jrxml** : le modèle du rapport, fichier XML texte, **le seul à maintenir et versionner**
- **.jasper** : le modèle compilé, produit automatiquement à partir du .jrxml
- **.jrprint** : le document généré avant export (interne)
- **.jrtx** : bibliothèque de styles partagée entre plusieurs rapports
- Data adapters (.jrdax / .xml) : description d'une connexion ou d'une source de données
- Ressources : images (PNG, JPG, SVG), polices (.ttf, jar de police), sous-rapports
- Sorties : PDF, HTML, XLSX, DOCX, CSV, ...

Rappels sur XML

- Un fichier .jrxml est un fichier XML **bien formé** :
 - un seul élément racine : <jasperReport>
 - des balises correctement imbriquées et fermées
 - des attributs entre guillemets
- Depuis la version 7, le fichier ne référence plus de schéma XSD ni d'espace de noms : la syntaxe est vérifiée par la librairie à la compilation
- Un commentaire en tête indique la version de Studio et de la librairie qui ont produit le fichier
- Chaque élément porte un identifiant uuid généré par Studio : ne pas le recopier d'un élément à l'autre
- L'onglet Source de Studio permet de lire et de modifier ce XML directement

Blank A4

```
<!-- Created with Jaspersoft Studio version 7.0.8.final using JasperReports Library version 7.0.8-6f896111 -->
```

```
<jasperReport name="Blank_A4" language="java" pageWidth="595" pageHeight="842" columnWidth="555" leftMargin="20"
rightMargin="20" topMargin="20" bottomMargin="20" uuid="ba6d0b4e-9d60-4d48-8f5d-4d88ab017bdd">
```

```
<query language="sql"><![CDATA[]]></query>
```

```
<background splitType="Stretch"/>
```

```
<title height="79" splitType="Stretch"/>
```

```
<pageHeader height="35" splitType="Stretch"/>
```

```
<columnHeader height="61" splitType="Stretch"/>
```

```
<detail>
```

```
<band height="125" splitType="Stretch"/>
```

```
</detail>
```

```
<columnFooter height="45" splitType="Stretch"/>
```

```
<pageFooter height="54" splitType="Stretch"/>
```

```
<summary height="42" splitType="Stretch"/>
```

```
</jasperReport>
```

Structure du fichier

Sous la balise racine, on trouve séquentiellement :

- Des éléments **déclaratifs**
- Des éléments **structurels** définissant des bandes horizontales d'une certaine hauteur
- A l'intérieur des bandes, des **composants d'affichage** issus de la palette

Éléments déclaratifs

- ***style**** : ***N définitions de styles***
- query? : 1 seule requête principale (SQL ou autre)
- ***field**** : ***N définition de champs***
- ***sortField**** : ***N champs de tri***
- ***filterExpression?*** : ***1 expression booléenne pour filtrer les données***
- ***parameter**** : ***N définitions de paramètres de rapport***
- ***variable**** : ***N définitions de variable***
- ***group**** : ***N définition de regroupement de données***
- ***subDataset**** : ***N références à des jeu de données (exemple : requêtes SQL)***
- ***property**** : N propriétés arbitraires globales au rapport
- ***import**** : N importation de packages java
- ***scriptlet**** : N références à des scriptlets (Code spécifique Java)
- ***template**** : N références à des gabarits de styles

Éléments structurels

- **background?** : 1 fond de page
- **title?** : 1 bande de titre, affiché une seule fois en début de rapport
- **pageHeader?** : 1 entête de page affichée à chaque début de page
- **columnHeader?** : 1 entête de colonne (pour les rapports multi- colonnes) affichée à chaque changement de colonne
- **detail*** : 1 bande détail. Affichée autant de fois qu'il y a de résultat dans la requête principale
- **columnFooter?** : 1 pied de colonne (pour les rapports multi- colonnes)
- **pageFooter?** : 1 pied de page
- **lastPageFooter?** : Le pied de page de la dernière page
- **summary?** : 1 résumé affiché en fin de document
- **noData?** : Texte alternatif quand il n'y a pas de données

Exemple de fichier .jrxml (syntaxe JasperReports 7)

```
<!-- Created with Jaspersoft Studio version 7.0.8.final using JasperReports Library version 7.0.8 -->
<jasperReport name="Commandes" language="java" pageWidth="595" pageHeight="842" columnWidth="555"
    leftMargin="20" rightMargin="20" topMargin="20" bottomMargin="20" uuid="ba6d0b4e-9d60-4d48-8f5d-4d88ab017bdd">
    <parameter name="P_CLIENT" class="java.lang.Integer"/>
    <query language="sql"><![CDATA[SELECT id, date_cmd, montant FROM commandes WHERE client_id = ${P_CLIENT}]]></query>
    <field name="id" class="java.lang.Integer"/>
    <field name="date_cmd" class="java.sql.Date"/>
    <field name="montant" class="java.math.BigDecimal"/>
    <background splitType="Stretch"/>
    <title height="79" splitType="Stretch">
        <element kind="staticText" uuid="..." x="0" y="20" width="555" height="30" fontSize="18.0" bold="true" hTextAlign="Center">
            <text><![CDATA[Liste des commandes]]></text>
        </element>
    </title>
    <pageHeader height="35" splitType="Stretch"/>
    <columnHeader height="20" splitType="Stretch">
        <element kind="staticText" uuid="..." x="0" y="0" width="100" height="20"><text><![CDATA[N° commande]]></text></element>
    </columnHeader>
    <detail>
        <band height="20" splitType="Stretch">
            <element kind="textField" uuid="..." x="0" y="0" width="100" height="20">
                <expression><![CDATA[${F{id}}]]></expression>
            </element>
            <element kind="textField" uuid="..." x="100" y="0" width="100" height="20" pattern="dd/MM/yyyy">
                <expression><![CDATA[${F{date_cmd}}]]></expression>
            </element>
        </band>
    </detail>
    <pageFooter height="54" splitType="Stretch">
        <element kind="textField" uuid="..." x="455" y="0" width="100" height="20" hTextAlign="Right">
            <expression><![CDATA["Page " + ${PAGE_NUMBER}]]></expression>
        </element>
    </pageFooter>
</jasperReport>
```

Les fichiers créés avec une version 6 sont convertis automatiquement à l'ouverture dans Studio 7.

Recommandations pour l'environnement

- Utiliser **Jaspersoft Studio 7.0.8** (mars 2026) ou la version alignée sur la librairie utilisée par votre application ou votre serveur
 - Un .jrxml enregistré en version 7 n'est plus lisible par une librairie 6.x : vérifier la version cible avant de commencer
- Un workspace dédié, un projet par famille de rapports
- Dans le projet : dossiers images/, polices/, sous-rapports/, data adapters partagés
- Versionner les .jrxml (fichiers texte) avec un outil de gestion de versions ; ne pas versionner les .jasper
- Une source de données de test représentative, distincte de la production
- Tester tôt dans le format de diffusion réel (PDF le plus souvent), pas seulement dans l'aperçu

Installation de JasperStudio

- Téléchargement sur community.jaspersoft.com (compte gratuit requis) : Windows (installateur), macOS, Linux
- Studio embarque sa propre machine Java : aucune installation de Java préalable
- Mémoire : 4 Go de RAM conseillés, 1 Go d'espace disque
- Premier lancement : choix du workspace, puis la perspective « Report Design »
- Également disponible sous forme de plugin pour un Eclipse existant

Documentation disponible

- Jaspersoft Studio User Guide : community.jaspersoft.com/documentation
- JasperReports Library : jasperreports.sourceforge.net (schéma, exemples, guide des fonctions)
- Sources et suivi des évolutions : github.com/Jaspersoft/jasperreports
- Dans Studio : aide contextuelle sur chaque propriété (vue Properties) et sur l'éditeur d'expressions

Découverte de l'interface

Vue générale

Explorateur de projet

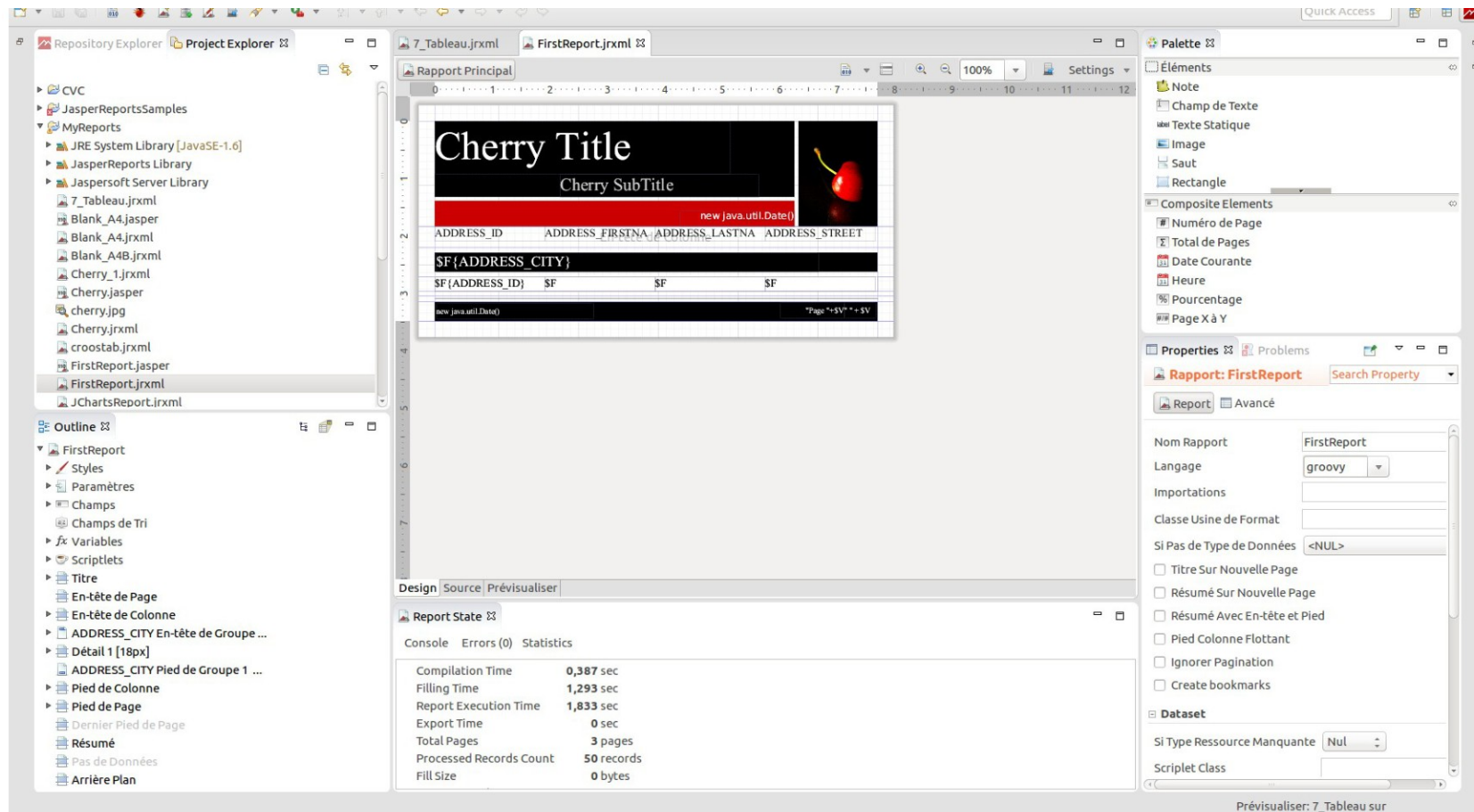
Zone de conception

Propriétés des objets

Aperçu et exécution

Repérage des éléments d'un rapport existant

Présentation JasperStudio



Vue générale de JasperStudio

- L'espace de travail de *jasperStudio* est multi-fenêtre :
 - **Espace de design central** permettant d'éditer les rapports, de visualiser le XML et de prévisualiser
 - **Outline** : Vue hiérarchique du rapport, permettant de sélectionner ses éléments
 - **Vue propriété** : Permet d'éditer les propriétés de l'élément sélectionné
 - **Palette** : Placer des éléments sur le rapport
 - **Outils de positionnement** : Alignement des éléments
 - **Problems** : Avertissements, Problème de parsing XML, de positionnement
 - **Report state** : Résultat des programmes internes (Compilation, fusion de données, ..)
 - **Explorateur du repository** JasperServer
- Accès à la configuration générale via *Windows*→ *Preferences*→ *JasperSoft Studio*

Explorateur de projet

- **Project Explorer** : projets, dossiers, fichiers .jrxml et ressources (images, polices, styles)
 - Double-clic sur un .jrxml : ouverture dans l'éditeur
 - Clic droit : nouveau rapport, import de fichiers, compilation, export
- **Repository Explorer** : data adapters définis dans le workspace, connexions à JasperReports Server
- Un fichier hors du projet peut être ouvert (File → Open File), mais ses ressources relatives ne seront pas trouvées
- Conseil : importer les rapports existants dans un projet avant de les modifier

Zone de conception

- Trois onglets en bas de l'éditeur : **Design**, **Source** (le XML), **Preview** (l'aperçu)
- En Design : les bandes du rapport, les règles, le zoom, la grille d'aimantation (Snap to grid)
- **Palette** : éléments de base (Static Text, Text Field, Image, Line, Rectangle, Frame, Break), éléments composites (Table, List, Chart, Crosstab)
- Glisser-déposer un champ, un paramètre ou une variable depuis l'Outline crée directement un champ texte
- Barre d'outils : alignement, distribution, taille, ordre d'affichage

Propriétés des objets

- Vue **Properties** : toutes les propriétés de l'élément sélectionné
 - Appearance : position, taille, couleurs, style appliqué
 - Borders : bordures et marges internes
 - Text / Static Text / Text Field : police, alignement, expression, pattern
 - Inheritance : d'où vient chaque valeur (élément, style, valeur par défaut)
 - Advanced : la liste complète des attributs, dont les propriétés d'export
- Sélection multiple : les propriétés communes sont modifiées d'un coup
- Sélectionner le rapport lui-même (clic dans le vide) : dimensions, marges, langage, colonnes

Aperçu et exécution

- Onglet **Preview** : compilation puis génération avec le data adapter choisi (liste déroulante)
- Choix du format de sortie : Java (écran), PDF, HTML, XLSX, DOCX, CSV, ...
- Saisie des paramètres d'entrée dans la vue **Input Parameters**
- Vue **Report State** : temps de compilation, de remplissage, d'export ; nombre de pages
- Vue **Problems** : erreurs de compilation (élément hors de la bande, expression invalide, ...)
- Console : détail des erreurs à l'exécution (image introuvable, requête SQL en erreur)

Repérer les éléments d'un rapport existant

- Vue **Outline** : arbre du rapport (styles, paramètres, champs, variables, bandes, éléments)
 - Un clic sur un élément de l'Outline le sélectionne dans la zone de conception
- Onglet Source + Ctrl+F : rechercher un libellé ou un nom de champ dans le XML
- Vue Properties : lire l'expression d'un champ texte pour savoir quelle donnée il affiche
- Menu Window → Show View pour réafficher une vue fermée ; Window → Reset Perspective pour tout remettre en place
- Vérifier le data adapter par défaut du rapport (onglet Design, barre du haut) avant tout aperçu

Test de la configuration

- Tester la configuration en créant un rapport vide :
 1. *File → New → Jasper Studio → JasperReports Project*
 2. *File → New → JasperReport*
 3. *Choix du gabarit « blank »*
 4. *Choix du nom de fichier.*
- Si tout est normal, *jasperStudio* génère un fichier *.jasper* et affiche une prévisualisation d'une page blanche

Atelier 1 : Démarrage de JasperStudio

- Installer et lancer JasperStudio, choisir un workspace
- Créer un projet et y importer un rapport existant
- Identifier les vues : Project Explorer, Outline, Properties, Palette, Problems
- Passer de Design à Source puis Preview
- Retrouver, dans un rapport existant, l'élément qui affiche une donnée donnée

Les sources de données

Logique générale d'alimentation d'un rapport

Sources supportées, sources JDBC

Visualiser les données dans JasperStudio

Paramètres, champs et variables

Logique générale d'alimentation d'un rapport

- Un rapport = un **modèle** (.jrxml) + des **données** fournies au moment de la génération
- Le modèle déclare des **champs** : le nom et le type de chaque colonne attendue
- Le **data adapter** décrit d'où viennent les données : connexion à une base, fichier, objets de l'application
- À la génération, le moteur parcourt les enregistrements un par un et remplit la bande Detail pour chacun
- Le même modèle peut être alimenté par des sources différentes : test dans Studio, production côté application

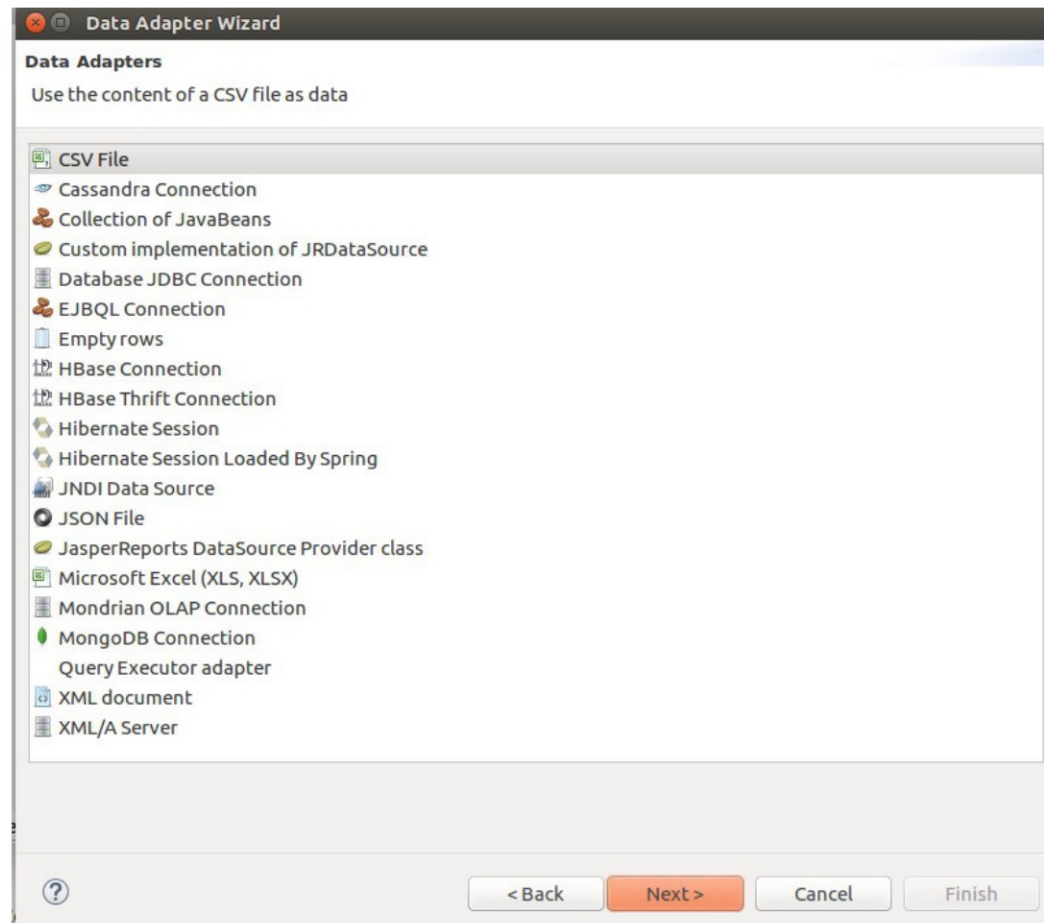
Un parcours séquentiel des enregistrements

- JasperReports lit les données par l'intermédiaire d'un **data adapter**, quelle que soit leur origine
- L'implémentation la plus utilisée est une base de données relationnelle interrogée en SQL via **JDBC**
- La source est parcourue ligne par ligne, du premier au dernier enregistrement, sans retour en arrière
- À chaque ligne, les champs `$F{}` prennent la valeur de la colonne correspondante et la bande Detail est remplie
- Conséquence : l'ordre d'affichage est celui de la requête, le tri se fait donc de préférence dans le SQL (`ORDER BY`)

Sources de données supportées

- **JDBC** : bases de données relationnelles (PostgreSQL, MySQL, Oracle, SQL Server, ...), la source la plus courante
- Fichiers : **CSV**, **Excel** (XLS, XLSX), **XML**, **JSON**
- Objets Java (JavaBeans) fournis par l'application : c'est souvent le cas en production
- **Empty data source** : aucune donnée, utile pour tester une mise en page ou un rapport statique
- JasperReports Server : data adapters distants
- Autres (Hibernate, OLAP, NoSQL, ...) selon la version et les extensions installées

File → New Data Adapter



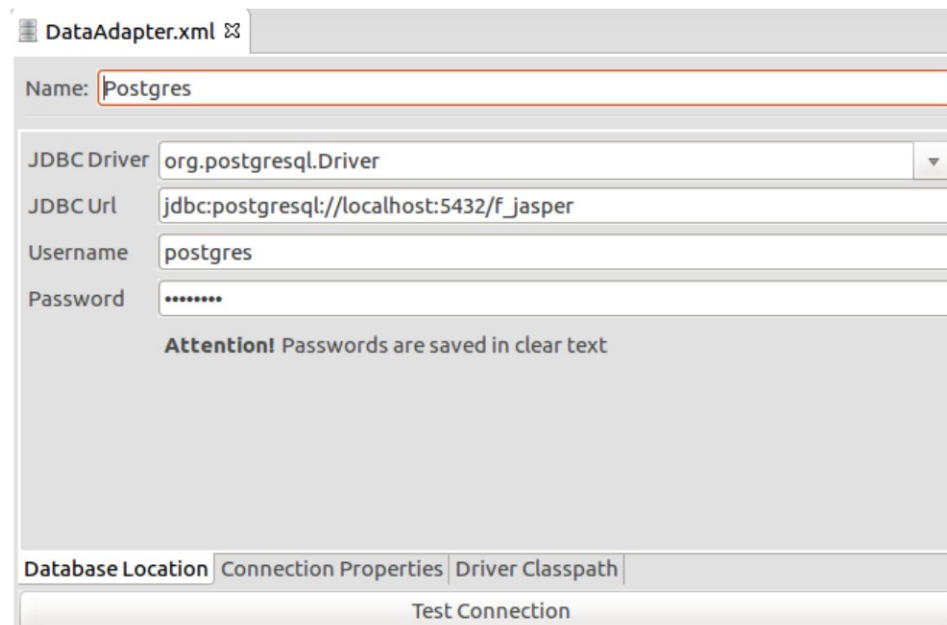
Rappels sur JDBC

- Une connexion JDBC permet à un programme Java de se connecter à une base de données relationnelles (toutes les bases sont supportées)
 - Pour cela, il est nécessaire d'utiliser les pilotes JDBC (bibliothèque Java) fournis par l'éditeur de base.
 - Configurer une connexion JDBC consiste à fournir :
 - La **classe principale** du Driver
 - L'**URL** d'accès à la base (syntaxe dépendante de l'éditeur)
 - Un **compte** base de données valide (login/mot de passe)
- jasperStudio* fournit les drivers JDBC pour *HyperSonic*.
- Les autres doivent être installés

Installation Driver

La bibliothèque du driver JDBC doit être trouvée par *jasperStudio*.

Lors de la création d'un DataAdapter, on lui indique où se trouve le .jar contenant le Driver



The screenshot shows the 'DataAdapter.xml' configuration window. The 'Name' field is set to 'Postgres'. The 'JDBC Driver' dropdown is set to 'org.postgresql.Driver'. The 'JDBC Url' is 'jdbc:postgresql://localhost:5432/f_jasper'. The 'Username' is 'postgres' and the 'Password' is masked with dots. A warning message states: 'Attention! Passwords are saved in clear text'. At the bottom, there are tabs for 'Database Location', 'Connection Properties', and 'Driver Classpath', with a 'Test Connection' button below them.

Name:	Postgres
JDBC Driver	org.postgresql.Driver
JDBC Url	jdbc:postgresql://localhost:5432/f_jasper
Username	postgres
Password	*****
Attention! Passwords are saved in clear text	
Database Location	Connection Properties
Driver Classpath	
Test Connection	

Dissociation rapport et source de données

- Il n'y a pas d'association stricte entre un rapport et une source de données.
- Lors de la génération, la source de données est fournie en paramètre
- Il est ainsi possible de réutiliser le même design sur différentes sources.
 - Exemple : Base de données de développement et base de données de production

Erreurs communes

- 3 types d'erreurs peuvent survenir lorsque l'on configure une connexion JDBC :
 - ***ClassNotFoundException*** : Le driver n'est pas dans le classpath
 - **L'URL** n'est pas correcte. Utiliser l'assistant de JasperStudio pour définir l'URL ou se référer au site de l'éditeur
 - Les **paramètres de connexion** ne sont pas corrects. Vérifier le nom de la base, l'utilisateur et le mot de passe.

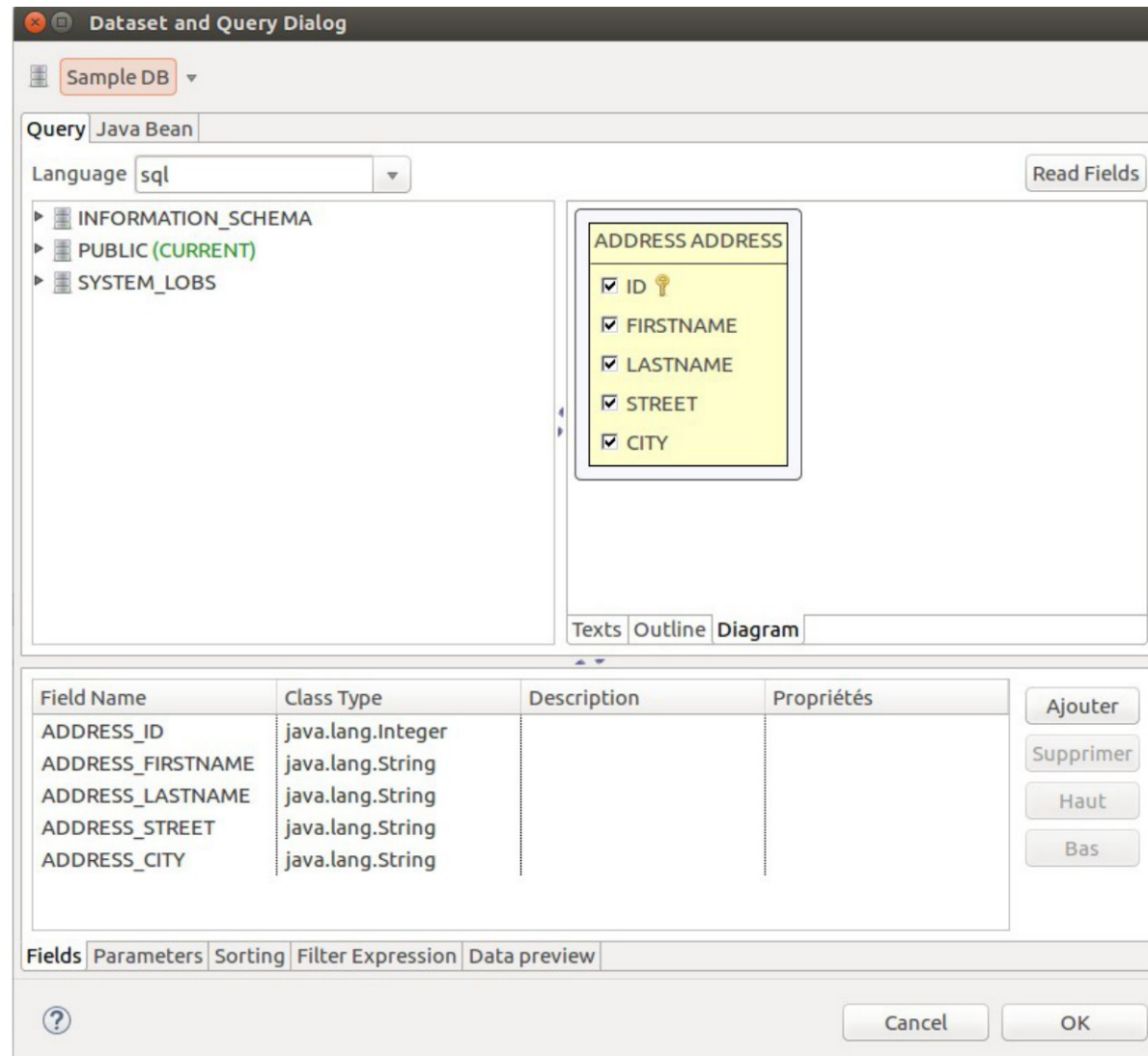
Requêtes SQL définies dans le rapport

- *jasperStudio* fournit plusieurs outils pour travailler avec JDBC et SQL
 - Test de la connexion JDBC
 - Découverte automatique des champs possibles avec leur type
 - Possibilité d'enregistrer automatiquement les champs
 - Un assistant pour mettre au point les requêtes (*Query Designer*)
 - Un prévisualiseur des données renvoyées par la requête

Visualiser les données dans JasperStudio

- Bouton **Test** du data adapter : vérifie la connexion
- Dialogue **Dataset and Query** (icône « base » de la barre d'outils) :
 - Onglet Query : saisie de la requête SQL, **Read Fields** pour créer les champs
 - Onglet Data preview : les premières lignes renvoyées par la requête
 - Outline SQL : liste des tables et colonnes de la base
- Onglet Preview du rapport : les données réelles dans la mise en page
- Les champs non utilisés apparaissent dans l'Outline : pratique pour les glisser dans le rapport

DataSet Dialog



Tri et filtre

- Les enregistrements récupérés d'une source de données peuvent être triés et filtrés programmatiquement par JasperReport
 - L'expression du filtre doit retourner un booléen
 - Le tri est basé sur un ou plusieurs champs en pouvant indiquer pour chaque la direction du tri
- Lorsque l'on utilise des bases de données relationnelles, il est en général plus performant d'utiliser les clauses SQL appropriées

Champs

- Les champs représentent l'unique façon de faire correspondre les données de la source avec les données du rapport.
- Les champs doivent être déclarés dans le rapport via l'élément **<field>** en indiquant
- L'attribut **name** : le nom de la colonne récupérée par la requêtes. Il doit être unique dans le rapport. Si plusieurs champs de la base ont le même nom, il faut utiliser des alias dans la requête SQL.
- L'attribut **class** : Indiquant la classe de l'objet pouvant être utilisé dans le rapport.
- Les éléments **<field>** sont placés sous l'élément **<jasperReport>**

Correspondance de type

Type SQL	Objet Java	Type SQL	Objet Java
CHAR	String	REAL	Float
VARCHAR	String	FLOAT	Double
LONGVAR CHAR	String	DOUBLE	Double
NUMERIC	java.math.BigDecim al	BINARY	byte[]
DECIMAL	java.math.BigDecim al	VARBINARY	byte[]
BIT	Boolean	LONGVARBINARY	byte[]
TINYINT	Integer	DATE	java.sql.Date

Exemple

```
<jasperReport>  
  <field name="id" class="java.lang.Integer"/>  
  <field name="name" class="java.lang.String"/>  
  <field name="city" class="java.lang.String">  
    <fieldDescription> Ville de naissance  
  </fieldDescription>  
  </field>  
  ....  
</japserReport>
```

Règles sur les champs

- Chaque champ doit correspondre alors à une colonne du *ResultSet*. C'est à dire avoir le **même nom et un type compatible** avec la colonne de la base.
 - Si le nom indiqué ne correspond pas à une colonne du *ResultSet*, une exception sera lancée au moment de l'exécution
 - Les colonnes présentes dans le *ResultSet* mais n'ayant pas de champs associés ne seront pas accessibles dans le rapport.

Paramètres, champs et variables

	Paramètre \$P{}	Champ \$F{}	Variable \$V{}
Origine	Fourni avant la génération (utilisateur, application)	Une colonne de la source de données	Calculé par le rapport pendant la génération
Change	Jamais pendant la génération	À chaque enregistrement	À chaque enregistrement, remise à zéro possible (page, groupe)
Exemples	Date de début, code client, titre, chemin des images	nom, montant, date_cmd	Total, compteur, numéro de page
Où le définir	Outline → Parameters	Dataset and Query → Read Fields	Outline → Variables
Usage type	Filtrer la requête SQL, personnaliser le rapport	Afficher la donnée	Sous-totaux, comptages, numérotation

Création de rapport

- *jasperStudio* propose un assistant pour créer un rapport. L'assistant est basé sur des **gabarits** (*template*) ; il est possible de définir ses propres gabarits
- L'assistant s'effectue en 5 étapes :
 - Sélection d'un modèle
 - Nom et emplacement du rapport
 - Source de données et requête SQL
 - Champs utilisés
 - Les groupes éventuels

Etape 2 : Fichier

The screenshot shows a Windows-style dialog box titled "Report". On the left, a "Steps" list contains seven items: "1. Choose Template", "2. Name and location" (which is bolded), "3. Query", "4. Fields", "5. Group by...", "6. Layout", and "7. Finish". The main area of the dialog is titled "Name and location" and contains three input fields. The first field is labeled "Report name:" and contains the text "My first report". The second field is labeled "Location:" and contains the text "C:\\devel\\reports"; to its right is a "Browse" button. The third field is labeled "File:" and contains the text "C:\\devel\\reports\\My first report.jrxml". At the bottom of the dialog, there are five buttons: "< Back", "Next >", "Finish", "Cancel", and "Help". The "Next >" button is highlighted with a blue border.

Report

Steps

1. Choose Template
- 2. Name and location**
3. Query
4. Fields
5. Group by...
6. Layout
7. Finish

Name and location

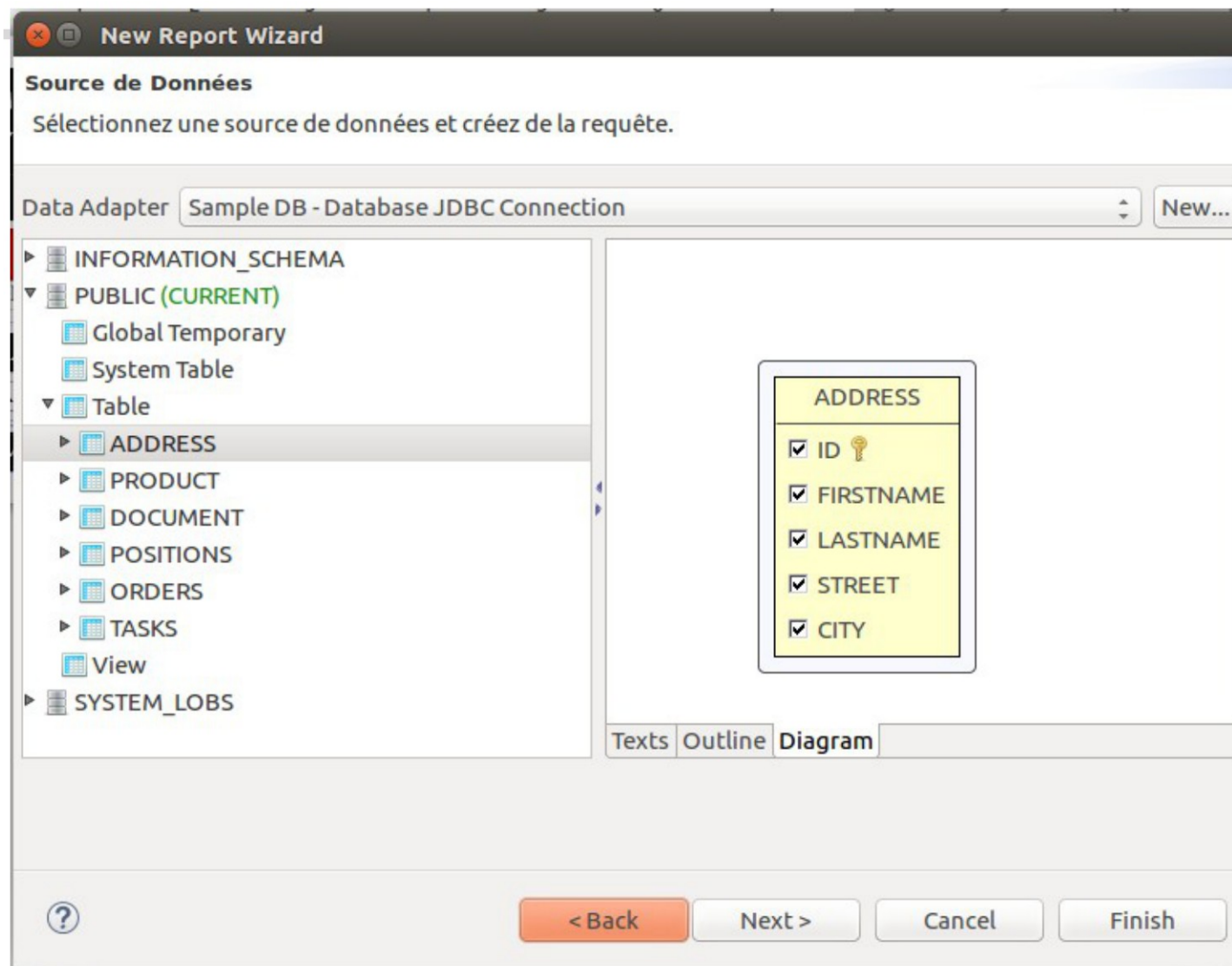
Report name:

Location:

File:

< Back Next > Finish Cancel Help

Etape 3 : Requête



Etape 4 : Champs

New Report Wizard

Champs

Veuillez sélectionner les champs d'ensemble de données

Champs d'Ensemble de Données	Champs
	ID
	FIRSTNAME
	LASTNAME
	STREET
	CITY

Navigation buttons: >, >>, <, <<, Haut, Bas, < Back, Next >, Cancel, Finish

Etape 5 : Groupe

New Report Wizard

Grouper Par

Veuillez sélectionner les champs à grouper par.

Champs d'Ensemble de Données		Champs	
ID	>	CITY	Haut
FIRSTNAME	>>		Bas
LASTNAME	<		
STREET	<<		

☐ Use the group fields as sort fields. Select this option if you want to aggregate all the group fields with the same value and not only the consecutive ones

? < Back Next > Cancel Finish

Atelier 2 : Premier rapport avec l'assistant

- Créer un data adapter JDBC vers la base de test et le tester
- Créer un rapport avec l'assistant : modèle, requête SQL, champs, sans groupe
- Observer les champs créés dans l'Outline et les bandes générées
- Prévisualiser en écran, puis en PDF et en Excel
- Modifier le titre et un libellé de colonne

Anatomie d'un rapport Jasper

Structure globale

Les bandes principales

Rôle de chaque bande dans le rendu final

Impact des bandes sur la pagination

Unité de mesure et attribut

- Les premières propriétés d'un rapport sont ses dimensions
- L'unité par défaut est le pixel et peut être changée
- Les attributs concernent la dimension de la zone de dessin :
 - Largeur et hauteur de page.
 - Largeur et hauteur des différentes marges

Bandes

- Un rapport est divisé en **bandes horizontales**.
 - La construction d'un rapport consiste à définir le contenu et les dimensions de ses bandes.
 - Chaque bande contient des éléments de rapport comme des lignes, des rectangles, des images ou des champs texte.
 - La structure d'un document Jasper est prédéfini avec un ensemble de bandes de départ
 - Les bandes peuvent être ajoutées/supprimées du rapport
 - Une bande correspond section à l'élément `<band>` dans le fichier *.jrxml*

Structure d'un rapport

- La structure d'un rapport est basée sur les bandes suivantes :
 - **<background>** : Fond de page appliqué sur toutes les pages
 - **<title>** : Titre apparaissant une seule fois au début
 - **<pageHeader>**, **<pageFooter>** : Entête et pied de page
 - **<columnHeader>**, **<columnFooter>** : Pour les rapports multi-colonnes, entête et pied des détails de colonne
 - **<groupHeader>**, **<groupFooter>** : Entête et pied d'un groupe (0 ou plusieurs)
 - **<detail>** : Zone répétée pour chaque donnée. Possibilité de définir plusieurs sections *<detail>*
 - **<lastPageFooter>** : Pied de page pour la dernière page
 - **<summary>** : Affiché à la fin du rapport
 - **<noData>** : Texte alternatif lorsqu'il n'y a pas de données

Bandes par défaut

	title	
	pageHeader	
	columnHeader	
	detail	
	columnFooter	
	pageFooter	
	summary	

Attributs d'une bande

- Attribut ***height***
 - C'est la hauteur de la bande dans la vue design. Cette hauteur peut varier lors de la fusion de données (si les données dépassent la hauteur spécifiée). C'est donc la hauteur minimale de la bande à la génération.
 - Lorsque cette valeur est modifiée, *JasperStudio* vérifie si la valeur est acceptable en fonction des éléments que la bande contient.
- Attribut ***printWhenExpression***
 - Expression booléenne permettant de cacher la bande
 - L'expression est évaluée à chaque fois que la bande est utilisée lors de la génération. (1 fois pour le titre, à chaque page pour l'entête de page, à chaque enregistrement pour le détail).
- Attribut ***splitType*** (anciennement ***isSplitAllowed***)
 - ***stretch***: La bande est divisée sur différentes pages seulement si son contenu dépasse la hauteur précisée.
 - ***prevent*** : La bande n'est pas divisée. (Seulement si sa taille dépasse la taille d'une page)
 - ***immediate*** : La bande peut être divisée.

Colonnes

- Les bandes pilotées par les enregistrements de la source de données (détail, groupe, sous-rapport, ...) peuvent être organisées en colonnes afin d'optimiser l'espace disponible.
- L'attribut ***columnCount*** du rapport permet donc de fixer le nombre de colonnes :
 - Le remplissage des colonnes peut alors être vertical ou horizontal (Orientation)
 - La somme des marges, des largeurs de colonnes (*columnWidth*) et de chaque espace entre les colonnes doit être inférieure à la largeur de la page, sinon des erreurs de compilation peuvent apparaître.
 - Lors du design, les éléments (champs, images, etc...) sont placés uniquement dans la première colonne.

Design

Listing des employés	
Listing des employés	
Nom	Prénom
\$F{FIRSTNAME}	\$F{FIRSTNAME}
Nom	Prénom
NOW()	

List of names on a two columns report

First name	Last name	First name	Last name
Laura	Steel	Sylvia	Fuller
Susanne	King	Susanne	Heiniger
Anne	Miller	Janet	Schneider
Michael	Clancy	Julia	Clancy
Sylvia	Ringer	Bill	Ott
Laura	Miller	Julia	Heiniger
Laura	White	James	Sommer
James	Peterson	Sylvia	Steel
Andrew	Miller	James	Clancy
James	Schneider	Bob	Sommer
Anne	Fuller	Susanne	White
Julia	White	Andrew	Smith
George	Ott	Bill	Sommer
Laura	Ringer	Bob	Ringer
Bill	Karsen	Michael	Ott
Bill	Clancy	Mary	King
John	Fuller	Julia	May
Laura	Ott	George	Karsen

Rôle de chaque bande dans le rendu final

- **Title** : une seule fois, en début de rapport (peut occuper une page entière : page de garde)
- **Page Header / Page Footer** : sur chaque page (logo, date, numéro de page)
- **Column Header / Column Footer** : au début et à la fin de chaque colonne, donc répétés à chaque page
- **Detail** : une fois par enregistrement, c'est le cœur du rapport
- **Summary** : une fois, à la fin (totaux généraux, mentions légales)
- **Background** : sous toutes les autres bandes, sur chaque page (filigrane, fond de page)
- **No Data** : affiché à la place du rapport si la requête ne renvoie rien (propriété When No Data Type)

Impact des bandes sur la pagination

- Hauteur disponible pour les données = hauteur de page – marges – Page Header – Page Footer – Column Header – Column Footer
- Une bande dont la hauteur dépasse l'espace disponible provoque une erreur de compilation
- La hauteur d'une bande est un **minimum** : elle s'étire si un champ texte s'agrandit
- Quand la bande suivante ne tient plus sur la page, une nouvelle page est créée
- Les propriétés Split Type (bande), Position Type et Stretch Type (éléments) contrôlent ce comportement : détail au jour 2

Atelier 3 : Rapport de type liste à 2 colonnes

- Partir du rapport de l'atelier 2 et passer le rapport en 2 colonnes (Columns = 2)
- Ajuster la largeur des colonnes et l'espacement, choisir le sens de remplissage
- Placer les éléments dans la première colonne seulement
- Utiliser Column Header pour les entêtes répétés
- Vérifier le rendu PDF sur plusieurs pages

Les éléments de base

Texte statique, texte dynamique

Image, ligne, rectangle, cadre

Positionnement et redimensionnement

Propriétés essentielles

Introduction

- Les éléments d'un rapport sont des objets d'affichage (Texte, Image, Forme, Graphique, ...) qui peuvent être dimensionnés, alignés, formatés etc.
- Tout contenu est créé via ces éléments.
- La palette de *jasperStudio* proposent un certain nombres d'éléments :
 - Ligne, Rectangle, Ellipse, Texte statique, Champ texte, Image, Cadre, Sous-rapport, Tableau, Liste, Tableau croisé, Graphique, Saut de page ou colonne
- Finalement, il est également possible d'étendre la bibliothèque d'éléments avec du code java

Hiérarchie des éléments

Les éléments sont organisés hiérarchiquement ; ils ont tous des propriétés communes et chaque niveau de la hiérarchie apporte ses spécificités

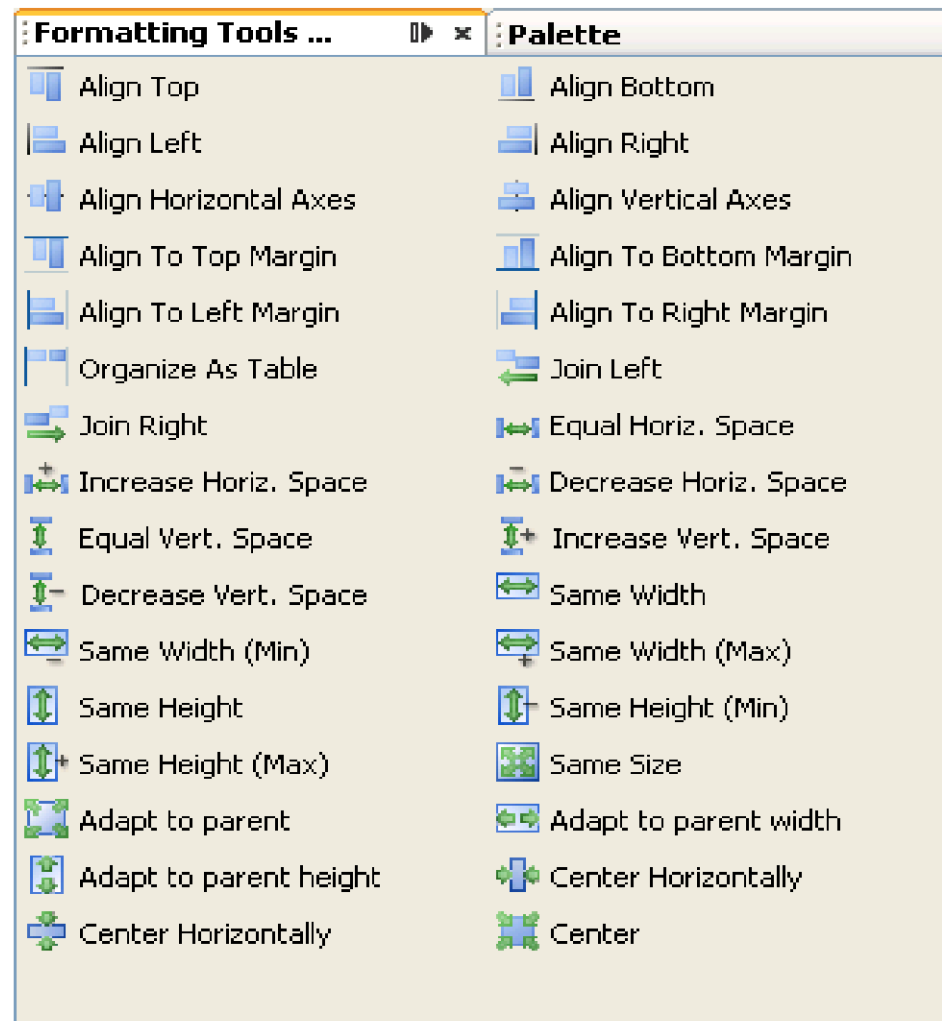
Timeseries Chart - Propriétés	
▼ Propriétés	
Left	0
Top	453
Width	555
Height	136
Forecolor	■ [0,0,0]
BackColor	□ [255,255,255]
Opaque	☐
Style	
Key	
Position Type	Fix Relative to Top
Stretch Type	No stretch
Print Repeated Values	☒
Remove Line When Blank	☐
Print In First Whole Band	☐
Print When Detail Overflows	☐
Print When Group Changes	
Print When Expression	
Properties expressions	No properties set
► Common chart properties	
▼ TimeSeriesPlot properties	
Show Lines	☒
Show Shapes	☒
Time Axis Label Expression	

Graphique: Time Series

Positionnement et taille

- Les éléments sont positionnés via leurs attributs ***top*** et ***left***
 - Ces valeurs sont relatives au container parent (bande ou frame)
- => Il est impossible de positionner un élément à cheval sur deux bandes
- La taille de l'élément est spécifié par les attributs ***width*** et ***height***

Outils de formatage



Outils de formattage (2)

Outil	Description	Multi
Align (left, right, top, bottom)	Aligne en fonction des côtés du 1er élément	X
Align Axis (horizontal, vertical)	Aligne en fonction des axes du 1er élément	X
Align band (top, bottom)	Aligne en fonction de la bande parente	
Same (width, height, size)	Fixe les dimensions par rapport au 1er élément	x
Same max (width,height)	Fixe les dimensions par rapport à l'élément le plus grand	x
Same min (width,height)	Fixe les dimensions par rapport à l'élément le plus petit	x
Center (horizontally, vertically)	Centre par rapport au container	

Outils de formattage (3)

Outil	Description	Multi
Center in background	Centre par rapport à la page et positionne dans le fond	
Join sides (right, left)	Juxtapose horizontalement les éléments	x
Horizontal space	Modifie les espaces horizontaux entre éléments	x
Vertical space	Modifie les espaces verticaux entre éléments	x
Adapt to parent	Redimensionne en fonction du parent	
Organize as table	Espace les éléments de façon égale	x

Axe z

- L'ordre des éléments d'une bande est important car il définit l'ordre d'impression. C'est le **z-index**.
 - Si les positions d'un élément se superposent avec celles d'un de ces prédécesseurs, il risque de le recouvrir lors de l'impression.
 - Dans d'autres situations, on peut prévoir deux éléments à la même position mais un seul est affiché grâce à l'attribut *printWhenExpression*.
 - Certains exporters, comme l'exporter HTML ne supportent pas les éléments superposés et donc les ignore.

Attributs des éléments

- **key** : Nom unique de l'élément
- **x, y, width, height** : Positions
- **foreground, background, opaque** : Couleurs
- **style** : Style de l'élément.
- **isRemoveLineWhenBlank** : Suppression de l'espace si l'élément est vide
- **properties** : Propriétés arbitraires de l'élément.

Type de position

- L'attribut ***PositionType*** détermine comment les coordonnées sont modifiées lorsque la hauteur de la bande change (lors de la fusion de données).
- 3 valeurs sont possibles :
 - ***FixRelativeToTop*** : Valeur par défaut, les coordonnées ne changent jamais.
 - ***Float*** : L'élément est progressivement déplacé vers le bas par les éléments précédents qui augmente leur taille
 - ***FixRelativeToBot*** : La distance entre l'élément et le bas de la bande reste constant.

Stretch Type : hauteur d'un élément quand la bande s'étire

- **NoStretch** (défaut) : la hauteur ne change pas
- **ElementGroupHeight** : l'élément prend la hauteur du groupe d'éléments qui l'entoure (ligne verticale simulant une bordure de tableau)
- **ElementGroupBottom** : le bas de l'élément suit le bas du groupe
- **ContainerHeight** / **ContainerBottom** : même chose par rapport à la bande ou au cadre conteneur
- Les anciennes valeurs `RelativeToBandHeight` et `RelativeToTallestObject` (version 6) sont converties automatiquement par Studio 7

Impression

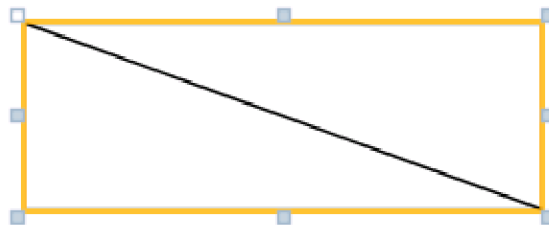
- ***isPrintRepeatedValues*** : Impression si valeur égale au précédent enregistrement
- ***isPrintInFirstWholeBand*** : Garantit l'impression de l'élément lors d'une nouvelle page ou nouvelle colonne.
- ***isPrintWhenDetailOverflows*** : Imprimé sur la page suivante, lorsque la bande ne tient pas sur la page courante
- ***printWhenGroupChanges*** : Imprimé lors d'une rupture de groupe
- ***printWhenExpression*** : Expression booléenne conditionnant l'impression

Éléments graphiques

- Les éléments graphiques sont :
 - Les ligne
 - Les rectangles
 - Les ellipses
- Ces éléments ont des attributs supplémentaires communs
 - **pen** : Le trait dessinant la forme
 - **fill** : Le remplissage de la forme

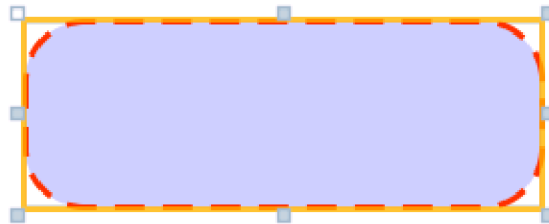
Ligne

- Une ligne est définie par la diagonale d'un rectangle
- L'attribut ***direction*** détermine la diagonale:
 - *TopDown*
 - *BottomUp*



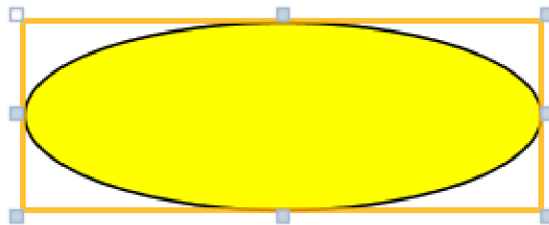
Rectangle / Rectangle arrondis

- Possibilité d'avoir des coins arrondis en utilisant l'attribut ***radius***



Ellipse

- Défini également à partir d'un rectangle, cet élément n'a pas d'attributs spécifiques



Cadre

- Un **cadre** (*frame*) est un élément qui contient d'autres éléments. Éventuellement, il peut définir une bordure
- On peut imbriquer autant de niveau que l'on veut.
- Par rapport à l'utilisation d'un rectangle, un cadre permet de :
 - Déplacer tous les éléments du cadre en même temps
 - Disposer automatiquement les éléments contenus via un layout particulier
 - Pas besoin de jouer sur l'axe Z

Images

- C'est l'objet le plus complexe des objets graphiques. Il sert à :
 - Insérer un fichier image binaire (JPEG, PNG, GIF)
 - Ou définir une zone de *Canvas*. Un code Java pourra alors dessiner le graphique ou produire une image binaire.

Textes

- Deux éléments sont prévus pour afficher du texte :
 - Les **textes statiques** dont la valeur ne change pas et est connue au moment du design :

<staticText>

- Les **champs textes** dont la valeur est fournie sous forme d'expression :

<textField>

Attributs

- Police : *jasperStudio* présente toutes les polices trouvées dans le système mais attention à la compatibilité au moment de la génération.
- Taille de police (entier), Gras et italique, Souligné, Barré
- Alignements horizontaux et verticaux
- Rotation : None, Left , Right, UpsideDown
- Espacement des lignes : Simple, 1.5 ou Double
- Markup : Utilisation d'un langage de balise (HTML ou RTF)

Attributs d'un champ texte

- **Blank When Null** : affiche une chaîne vide au lieu de « null »
- **Text Adjust** : comportement quand le texte dépasse la taille de l'élément
 - CutText : le texte est coupé (défaut)
 - **StretchHeight** : l'élément s'agrandit en hauteur, la bande aussi
 - ScaleFont : la taille de police est réduite pour tenir
- **Pattern** : format d'affichage des nombres et des dates, ex.
dd/MM/yyyy ou #, ##0.00
- **Evaluation Time** : moment où l'expression est calculée (Now, Report pour un « Page x sur y », Group pour un total de groupe)
- Markup : interprétation de balises (HTML simplifié, styled) dans le texte

Bordures et marges internes

- Onglet **Borders** de la vue Properties
- Chaque côté se règle séparément : épaisseur, style (plein, pointillé, double), couleur
- **Padding** : marge interne entre la bordure et le texte, par côté
- Un tableau se dessine avec les bordures des champs texte, plutôt qu'avec des lignes séparées
- Pour que les bordures suivent un texte qui s'étire : même Stretch Type sur tous les éléments de la ligne, ou utiliser un cadre (Frame)
- Les bordures se définissent de préférence dans un style pour rester homogènes

Propriétés essentielles : synthèse

- Taille et coordonnées : x, y, width, height, en pixels à 72 dpi (1 cm $\approx$ 28 px)
- Alignement du texte : horizontal (gauche, centre, droite, justifié) et vertical (haut, milieu, bas)
- Bordures et marges internes : onglet Borders
- Redimensionnement : Text Adjust pour le texte, Stretch Type pour suivre la bande, Position Type pour se décaler
- Print When Expression : afficher l'élément sous condition
- Remove Line When Blank : libérer l'espace vertical si l'élément est vide

Paramètres

- Les **paramètres** sont des données passées lors de la génération du document dynamique qui ne proviennent pas de la source de données:
 - Par exemple des données de présentation comme le nom de l'utilisateur, le titre du rapport, ...
 - Des paramètres de la requête SQL. Les paramètres agissent alors comme des filtres sur les données récupérées. Exemple : une période, un montant, etc ...
 - Des paramètres servant à l'évaluation d'expression

Déclaration

- La déclaration d'un paramètre dans un design consiste à spécifier son **nom** et son **type** (classe Java):
- Dans le fichier *.jrxml*, cela donne :
<parameter **name**="ReportTitle" **class**="java.lang.String"/>
<parameter **name**="MaxOrderID" **class**="java.lang.Integer"/>
<parameter **name**="SummaryImage" **class**="java.awt.Image"/>

Attributs additionnels

- L'élément ***defaultValueExpression*** permet de spécifier une valeur par défaut. Dans cette expression, il n'est pas possible d'utiliser des champs ou variables.
- Les attributs ***IsForPrompting***, ***Description*** et ***properties*** ne sont pas utilisés directement par JasperReport mais peuvent être utilisés par d'autres applications externes :
 - pour déterminer si ce paramètre doit être fourni interactivement.
 - Une documentation
 - Stockée un ensemble de propriétés liées au paramètre. Par exemple, la description du champ en différents langages

Utilisation dans une requête

- Les paramètres peuvent être utilisés dans les requêtes SQL :

- Comme paramètres normaux d'un *java.sql.PreparedStatement* :

- SELECT * FROM Orders WHERE CustomerID = **\$P{OrderCustomer}***

- Ou comme tout ou partie de la requête SQL

- SELECT * FROM **\$P!{TablesClause}***

Clause IN et NOT IN

- *JasperReports* propose une syntaxe spécifique à utiliser dans les clauses IN et NOT IN des requêtes SQL. Exemple :

```
SELECT * FROM ORDERS WHERE $X{IN,  
SHIPCOUNTRY,myCountries}
```

- La clause `$X{}` prend 3 paramètres :
 - Le type de la fonction à appliquer (IN ou NOT IN)
 - Le nom du champ à évaluer
 - Le nom du paramètre

Atelier 4 : Mise en forme de type tableau, paramètres de requête

- Construire un tableau : entêtes en Column Header, données en Detail, bordures et alignements
- Ajouter un paramètre (période, client) et l'utiliser dans la clause WHERE de la requête
- Donner une valeur par défaut au paramètre et le saisir dans l'aperçu
- Afficher la valeur du paramètre dans le titre du rapport
- Vérifier le rendu en PDF

Jour 2 : Formatage de rapport

Gestion des images et logos

Gestion des polices

Styles et formats

Contrôler la pagination

Expressions simples

Gestion des images et logos

Insertion et remplacement d'un logo

Dimensions et proportions

Positionnement dans l'en-tête

Image en filigrane

Qualité de rendu

Erreurs courantes

ImageExpression

- Les images ne sont pas stockées avec le rapport, seule sa référence via l'élément **<imageExpression>** y est présente.
- L'attribut **class** de la balise indique le type de l'expression ; les principales valeurs sont :
 - **java.lang.String** : Emplacement et nom du fichier. Ex : */home/public/images/logo-footer.gif*
 - **java.io.File** : Objet Java File. Ex : *new java.io.File("c:\\myImage.jpg")*
 - **java.net.URL** : L'URL de l'objet Ex : *new java.net.URL("http://127.0.0.1/test.jpg")*

Portabilité

- Lorsque l'on définit une image en précisant son emplacement, il faut que cet emplacement soit valide dans l'environnement de développement **ET** de production
- Les meilleures pratiques pour éviter les problèmes en production sont :
 - Utiliser un **paramètre** dans l'expression indiquant le répertoire où sont placées les images : **`$P{MY_IMAGES_DIRECTORY} + "myImage.png"`** Au moment de la génération, l'application métier ou JasperServer positionne ce paramètre.
 - Utiliser le **classpath** JAVA. (Recommandé) Dans ce cas, l'expression de l'emplacement est relative au classpath et le déploiement de l'application inclut les images dans l'archive de l'application.

Ces méthodes sont utilisées pour les autres types de ressources externes (sous-rapport, police externe, scriptlet, ressource bundle, ...)

Attributs de chargement

- ***isLazy*** : L'image n'est chargée qu'à l'exécution du rapport
- ***isUsingCache*** : L'image est gardée en mémoire pour être réutilisée
- ***onErrorType*** : Spécifie le comportement si l'image ne peut pas être chargée :
 - ***Error*** : Une exception Java est lancée.
 - ***Blank*** : L'exception est ignorée et l'image est remplacée par un rectangle blanc.
 - ***Icon*** : L'exception est ignorée et un icône de remplacement est affiché.

Autres attributs

- **Scale** : Définit comment l'image se comporte par rapport aux dimensions de l'élément :
 - **Clip** : L'image n'est pas modifiée, et seule la partie correspondant aux dimensions de l'élément est affichée
 - **FillFrame** : L'image remplit le cadre
 - **RetainShape** : L'image est adaptée en conservant les proportions
 - **RealHeight** : Conserve la hauteur réelle de l'image (*JRRenderable*)
 - **RealSize** : Conserve la taille réelle de l'image (*JRRenderable*)
- Alignements verticaux et horizontaux (**halign** et **valign**)

Insérer et remplacer un logo

- Copier le fichier image dans le projet (dossier images/) avant de l'insérer
- Glisser l'élément **Image** de la palette dans la bande Page Header, choisir le fichier
- L'expression de l'image contient le chemin : "images/logo.png" (relatif au rapport) ou un paramètre
- Remplacer un logo : remplacer le fichier en gardant le même nom, ou modifier l'expression
- En production, l'image doit être livrée avec le rapport : même chemin relatif, ou paramètre fixé par l'application
- Éviter les chemins absolus (C:\Users\...) : ils ne fonctionnent que sur le poste du concepteur

Dimensions et proportions

- L'élément a sa propre taille ; l'image est adaptée selon **Scale Image** :
 - **RetainShape** : proportions conservées, l'image tient dans l'élément (choix conseillé pour un logo)
 - FillFrame : l'image remplit l'élément et se déforme
 - Clip : taille réelle, la partie qui dépasse est coupée
 - RealHeight / RealSize : l'élément prend la taille réelle de l'image
- Alignement horizontal et vertical de l'image dans l'élément
- Régler la taille de l'élément aux proportions de l'image pour éviter les espaces blancs

Positionnement précis dans l'en-tête

- Saisir x, y, width, height dans la vue Properties plutôt qu'à la souris
- Grille et aimantation (View → Snap to grid), guides depuis les règles
- Outils d'alignement : aligner à gauche, centrer dans la bande, répartir
- Un élément ne doit pas dépasser sa bande : erreur « element is outside the band »
- Ajuster la hauteur de la bande Page Header à l'image la plus haute
- Position Type : FixRelativeToTop pour un logo d'en-tête

Image en filigrane

- Placer l'image dans la bande **Background** : elle est dessinée sous le contenu de chaque page
- Donner à la bande la hauteur de la page et centrer l'image
- Utiliser un PNG avec transparence ou une image très claire : le texte doit rester lisible
- Un texte statique en gros caractères gris clair (« BROUILLON », « CONFIDENTIEL ») peut jouer le même rôle, avec une rotation
- Afficher le filigrane sous condition avec Print When Expression, par exemple un paramètre \$P{BROUILLON}
- Le fond n'est pas exporté dans tous les formats (Excel notamment)

Qualité de rendu à l'écran et à l'impression

- Le rapport est conçu en pixels à 72 dpi ; l'impression se fait à 300 dpi ou plus
- Fournir des images d'une résolution suffisante (au moins 3 à 4 fois la taille affichée) : le PDF conserve l'image d'origine
- **SVG** : format vectoriel, net à toutes les tailles, idéal pour un logo
- Éviter d'agrandir une petite image : pixellisation
- Un logo lourd dans Page Header est répété sur chaque page : privilégier un fichier optimisé
- Vérifier le rendu dans le format final, l'aperçu écran ne suffit pas

Erreurs courantes avec les images

- **Image déformée** : Scale Image = FillFrame → passer en RetainShape
- **Image tronquée** : Scale Image = Clip ou élément trop petit → agrandir l'élément ou changer le mode
- **Mauvais ancrage** : l'image se décale quand la bande s'étire → vérifier Position Type et Stretch Type
- **Espace vide à la place de l'image** : chemin introuvable et On Error Type = Blank → mettre Error pendant la conception pour voir le message
- **Image présente dans Studio, absente en production** : chemin absolu ou fichier non livré
- **Image qui déborde sur l'élément voisin** : hauteur de bande insuffisante

Atelier 5 : Adaptation logo, images de background

- Remplacer le logo d'un rapport existant par le logo de l'entreprise, sans déformation
- Positionner le logo et le titre dans l'en-tête de page
- Ajouter un filigrane « BROUILLON » conditionné par un paramètre
- Comparer le rendu PDF et Excel

Gestion des polices

Polices standard et polices spécifiques

Éditeur vs PDF

Import d'une police

Taille, graisse, style, interlignage

Bonnes pratiques

Polices

Java et donc JasperReport distingue 2 types de Police :

Les polices physiques installées sur le

- système

Les polices logiques : *Serif*, *SansSerif*,

- *Monospaced*, *Dialog* et *DialogInput*.

Elles n'ont pas de fichier installé sur le système mais elles sont associés à une des polices physique.

La correspondance peut dépendre du système !!

Polices standard et polices spécifiques

- Le PDF connaît nativement 14 polices standard : Helvetica, Times, Courier et leurs variantes
- Les autres polices doivent être **embarquées** dans le PDF, sinon le lecteur en substitue une autre
- JasperReports fournit des **extensions de police** prêtes à l'emploi : DejaVu Sans, DejaVu Serif, DejaVu Sans Mono
- Une police d'entreprise (Arial, Calibri, police de la charte) doit être ajoutée comme extension
- Les polices logiques Java (SansSerif, Serif) dépendent du système : à éviter dans les rapports

La police fonctionne dans l'éditeur mais pas dans le PDF

- L'aperçu écran utilise les polices installées sur le poste : tout semble correct
- L'export PDF ne peut embarquer que les polices déclarées en **extension de police**
- Symptômes : police remplacée par Helvetica, gras ou italique perdus, caractères accentués ou spéciaux manquants, erreur « Font ... is not available to the JVM »
- Cause fréquente : nom de police tapé à la main dans les propriétés, non déclaré dans Studio
- Solution : déclarer la police dans Studio (extension), l'exporter en jar et livrer ce jar avec le rapport

Bonne pratique

L'utilisation de police système comporte toujours le risque qu'une police utilisée lors du développement ne soit pas présente dans l'environnement de production.

La bonne pratique est d'embarquer les fichiers de polices dans l'archive déployée.

Cette technique appelé « **Font Extension** » nécessite de fournir un fichier jar contenant les fichiers polices, un fichier *properties* et un descripteur XML associant les fichiers polices aux locales et style de police (italique, gras, ...)

Le jar peut être généré directement à partir de *jasperStudio* après avoir ajouté la police voulue

Voir l'exemple dans */demo/samples*

Importer une police dans JasperStudio

- Window → Preferences → Jaspersoft Studio → **Fonts** → Add
- Renseigner le nom de la famille et les fichiers .ttf : normal, gras, italique, gras italique
- Cocher **Embed this font in PDF document** ; encodage PDF Identity-H pour les accents
- Ajouter les locales concernées si la police ne couvre pas tous les alphabets
- La police apparaît alors dans la liste des polices des éléments
- **Export** : produit un jar à déposer dans l'application ou à charger dans JasperReports Server ; sans ce jar, la production retombe sur une police de substitution

Taille, graisse, style, interlignage

- Onglet Text de la vue Properties : famille, taille, gras, italique, souligné, barré
- Alignement horizontal et vertical, rotation du texte
- **Interlignage** (Line Spacing) : simple, 1,5, double, proportionnel ou fixe
- Paragraphe : retrait de première ligne, espacement avant et après, tabulations
- La hauteur de l'élément doit suivre la taille de police : 12 pt de texte demandent environ 15 px
- Markup « styled » pour mettre en gras une partie seulement d'un champ texte

Bonnes pratiques pour éviter les écarts de rendu

- Une seule famille de police par rapport, déclarée en extension et définie dans le style par défaut
- Ne jamais dépendre des polices installées sur le poste de conception
- Tester l'export PDF dès le début, puis à chaque changement de police
- Livrer et versionner le jar de police avec les rapports
- Vérifier les caractères spéciaux utilisés (€, œ, guillemets typographiques) avec la police choisie
- Excel et HTML utilisent les polices du poste du lecteur : rester sur des polices répandues pour ces formats

Atelier 6 : Déploiement d'une police spécifique

- Déclarer une police TrueType dans JasperStudio avec ses quatre variantes
- L'appliquer au rapport et vérifier le rendu PDF (accents, gras)
- Exporter le jar de police et le livrer avec le rapport
- Reproduire le cas d'erreur d'une police non déclarée et l'interpréter

Styles et formats

Intérêt des styles

Appliquer un style commun

Formats de données

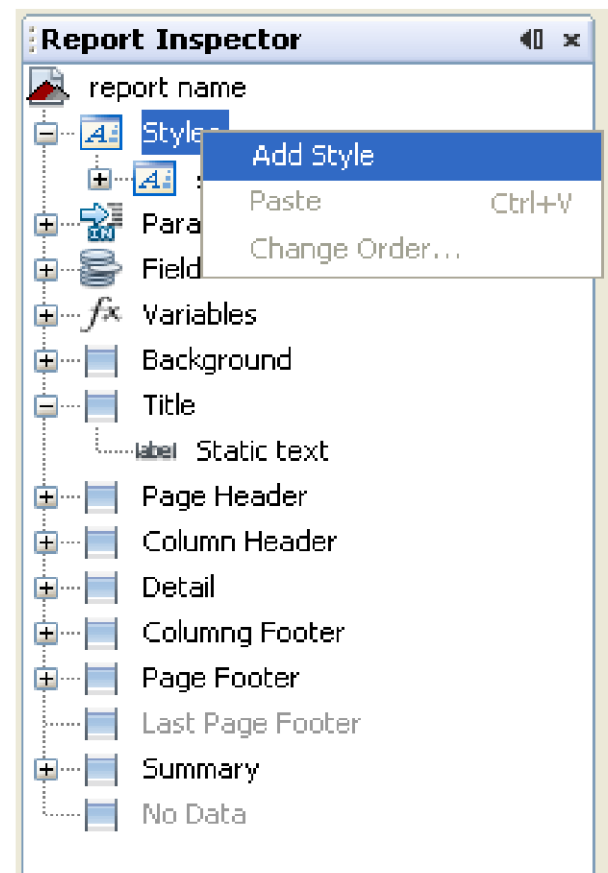
Règles de lisibilité

Titres, sous-titres, données et totaux

Styles

- Les styles sont utilisés pour regrouper un ensemble de propriétés de présentation et peuvent ensuite être appliqués aux éléments du rapport
- => Ainsi, la modification d'un style peut affecter un ensemble d'éléments du rapport
- Les éléments peuvent individuellement surcharger les propriétés définies par leur style
 - Il est possible de définir un style par défaut au niveau du rapport.

Ajout de style



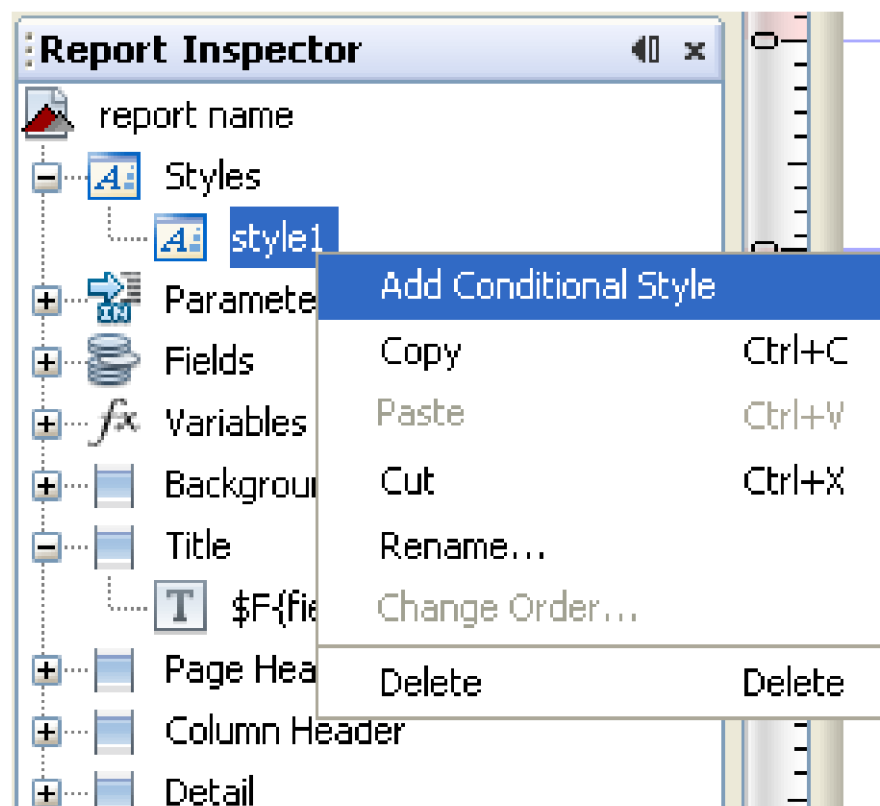
Appliquer un style commun à plusieurs éléments

- Outline → Styles → clic droit → Create Style : nom, police, taille, couleurs, bordures, alignement
- Un style peut hériter d'un autre (Parent Style) : « Total » hérite de « Données » et ajoute le gras
- Cocher **Default Style** pour qu'il s'applique à tout élément sans style
- Appliquer : sélection multiple des éléments → propriété Style
- Modifier le style modifie tous les éléments d'un coup
- Un élément peut surcharger localement une propriété du style ; l'onglet Inheritance montre d'où vient chaque valeur

Styles conditionnels

- Il est possible de créer des rapports où le style appliqué change dynamiquement en fonction de variable ou d'expression du rapport *Exemple : Mettre en rouge un texte en fonction d'une condition*
- *jasperStudio* permet d'associer à un style, un **style conditionnel** qui lorsque certaines conditions seront remplies surchargera certaines propriétés du style
- La condition peut s'exprimer en fonction de toutes les propriétés du rapport (paramètres, champs, variables, ...)

Ajout de style conditionnel



Librairie de styles

Au lieu de définir les styles indépendamment dans chaque rapport.

Il est possible de référencer une librairie de style dans un rapport

La librairie de styles (**Style Template File**) est un fichier **.jrtx** complètement autonome et ayant un éditeur spécifique

Formats de données : les patterns

- Propriété **Pattern** du champ texte, avec un éditeur graphique dans Studio ; le champ doit être de type numérique ou date, pas texte

Donnée	Pattern	Valeur	Affichage
Entier avec séparateur de milliers	<code>#,##0</code>	1234567	1 234 567
Montant	<code>#,##0.00 €</code>	1234.5	1 234,50 €
Pourcentage	<code>#,##0.0 %</code>	0.1534	15,3 %
Date courte	<code>dd/MM/yyyy</code>	2026-09-12	12/09/2026
Date longue	<code>EEEE d MMMM yyyy</code>	2026-09-12	samedi 12 septembre 2026
Date et heure	<code>dd/MM/yyyy HH:mm</code>		12/09/2026 14:30
Nombre sans décimale forcée	<code>#,##0.##</code>	12.5	12,5

Locale et formats

- Les séparateurs (virgule, espace, point) et les noms de jours dépendent de la **locale** du rapport
- Paramètre prédéfini \$P{REPORT_LOCALE} : fixé par Studio à l'aperçu, par l'application ou le serveur en production
- Un rapport correct dans Studio peut afficher « 1,234.50 » en production si la locale du serveur est anglaise
- Forcer la locale dans les propriétés du rapport ou la passer en paramètre
- Le pattern s'écrit toujours avec les symboles anglais (, milliers, . décimale) ; la locale fait la traduction
- Pour un format vraiment fixe : les fonctions TEXT() et DATEFORMAT() dans l'expression

Règles de lisibilité pour un public métier

- Une hiérarchie visuelle claire : 3 niveaux maximum (titre, entêtes, données)
- Deux tailles de police et une seule famille suffisent ; le gras pour les entêtes et totaux
- Nombres alignés à droite, textes à gauche, même nombre de décimales dans une colonne
- Unités et devise dans l'entête de colonne plutôt que sur chaque valeur
- Aérer : marges internes, interligne, filets fins plutôt que grilles complètes
- Lignes alternées avec un style conditionnel : $\$V\{\text{REPORT_COUNT}\} \% 2 == 0$
- Contraste suffisant pour l'impression noir et blanc : pas de texte gris clair sur fond gris

Différencier titres, sous-titres, données et totaux

- Une charte de styles type, à créer une fois dans une bibliothèque .jrtx et à réutiliser dans tous les rapports

Style	Usage	Police	Autres propriétés
Titre	Bande Title	18 pt, gras	Centré ou aligné au logo
Sous-titre	Période, filtres appliqués	12 pt, italique	Sous le titre, gris foncé
Entête de colonne	Column Header	10 pt, gras	Fond gris clair, filet bas 1 pt
Données	Detail	9 pt	Nombres à droite, padding 2 px
Total	Summary, pieds de groupe	9 pt, gras	Filet haut 1 pt, hérite de Données
Pied de page	Page Footer	8 pt	Numéro de page, date d'édition, gris

Atelier 7 : Mise en place d'une charte graphique

- Créer les styles Titre, Sous-titre, Entête, Données, Total avec héritage
- Les appliquer aux éléments du rapport, définir le style par défaut
- Formater les montants, pourcentages et dates avec des patterns
- Ajouter un style conditionnel pour les lignes alternées et les montants négatifs
- Exporter les styles en bibliothèque .jrtx et la référencer depuis un second rapport

Contrôler la pagination

Fonctionnement des sauts de page

Hauteur disponible par page

Impact des bandes sur le rendu

Répétition des en-têtes

Éviter les coupures inopinées

Fonctionnement des sauts de page

- Le moteur remplit la page bande après bande ; quand la bande suivante ne tient plus, il ferme la page (Page Footer) et en ouvre une nouvelle (Page Header, Column Header)
- Saut de page explicite : élément **Break** de la palette (type Page ou Column), placé dans une bande
- Propriétés du rapport : **Title on a new page, Summary on a new page, Summary with page header and footer**
- Le saut de page dépend donc des hauteurs réelles après étirement, pas des hauteurs de conception
- Propriété du rapport **Ignore pagination** : tout sur une seule page, utile pour l'export Excel

Hauteur disponible par page

- A4 portrait : 842 px de haut ; marges haut et bas 20 px chacune → 802 px utiles
- Moins Page Header (60) et Page Footer (30), Column Header (25) et Column Footer (0) → 687 px pour les données
- Avec une bande Detail de 20 px : 34 enregistrements par page
- La somme des bandes non répétées (Title, Detail, Summary) ne doit pas dépasser l'espace utile : sinon erreur à la compilation
- Chaque champ en Stretch Height peut allonger la bande Detail : le nombre de lignes par page varie
- Vue Report State après l'aperçu : nombre de pages générées

Impact des bandes sur le rendu

- Page Header et Page Footer : chaque page, y compris la page de titre sauf si Title on a new page
- **Last Page Footer** : remplace Page Footer sur la dernière page (mentions finales, total général)
- Column Header : répété au début de chaque page et de chaque colonne
- Summary : après le dernier enregistrement, sur la même page s'il reste de la place
- Une bande vide (hauteur 0) ne consomme rien : supprimer celles qui ne servent pas
- Print When Expression sur une bande : la masquer pour certaines pages, ex. `$V{PAGE_NUMBER} > 1`

Répétition des en-têtes

- Column Header : répétition automatique sur chaque page pour un rapport de type liste
- Entête de groupe : propriété **Reprint Header On Each Page** du groupe
- Élément Table : les en-têtes de colonnes se répètent à chaque page ; l'en-tête de table (Table Header) ne s'affiche qu'une fois
- Sous-rapport : ses propres Column Header sont répétés dans la page du rapport parent
- Éviter de mettre les libellés de colonnes dans Title ou Page Header : ils ne suivraient pas les données

Éviter les coupures inopinées

- **Split Type** de la bande Detail : Prevent pour ne jamais couper un enregistrement sur deux pages
- Groupe : **Keep Together** pour garder l'en-tête de groupe avec sa première ligne, **Min Height To Start New Page**
- **Print When Detail Overflows** : réimprimer un élément (entête de bloc) si la bande déborde sur la page suivante
- Un seul mode d'étirement cohérent sur une ligne : Text Adjust Stretch Height sur les textes, Stretch Type sur les bordures et lignes
- Éviter Position Type Float mélangé à des éléments fixes
- Textes longs : prévoir la hauteur, ou accepter Stretch Height mais tester avec les données les plus longues

Page de garde et numérotation

- Page de garde : bande Title avec **Title on a new page** ; le Page Header n'y apparaît pas
- Numéro de page : champ texte `$V{PAGE_NUMBER}` dans Page Footer
- « Page 3 sur 12 » : deux champs texte, le premier avec Evaluation Time = Now, le second (total) avec Evaluation Time = **Report**
- Ou en une seule expression avec la fonction "Page " + `$V{PAGE_NUMBER}` et un second champ Report
- Date d'édition : `TODAY()` ou `new java.util.Date()` avec un pattern de date
- Recommencer la numérotation par groupe : variable `PAGE_NUMBER` remise à zéro n'est pas possible, utiliser un groupe avec Start New Page et une variable personnalisée

Atelier 8 : Page de garde et gestion des sauts de page

- Créer une page de garde avec logo, titre et paramètres du rapport
- Ajouter « Page x sur y » et la date d'édition en pied de page
- Empêcher la coupure d'un enregistrement et répéter les entêtes
- Forcer un saut de page avant le résumé et vérifier le nombre de pages

Expressions simples

Concaténer des champs
Afficher conditionnellement
Gérer une valeur vide ou nulle
Une logique de formule Excel
Libellés plus parlants

Expressions

- Les **expressions** sont utilisées pour exprimer la valeur de certains éléments du rapport : variables, critère de regroupement, ou même tout simplement paramètre visuel du rapport
- Elles peuvent utiliser des opérateurs et les méthodes du langage
- Les langages disponibles sont Java, Javascript et Groovy
- Les expressions peuvent faire référence à des paramètres, des champs ou des variables

Spécification des expressions

- La notation **`${ }`** permet de faire référence à des champs, des variables, des paramètres ou des clés de ressources
 - **`$F{ <fieldName> }`** pour un champ
 - **`$V{variableName}`** pour une variable
 - **`$P{parameterName}`** ou **`$P!{parameterName}`** pour un paramètre.
 - **`$R{resourceKey}`** pour la localisation

Types des expressions

- Le type de l'expression est souvent déterminé par son contexte d'utilisation (condition, calcul, texte, ...)
 - Si le langage est Java, les types supportés sont : *Boolean, Byte, Integer, Long, Float, Double, String, Date* et *Object*
 - En Javascript et Groovy, les expressions ne sont pas typées

Opérateurs

Opérateur	Description
+	Ajout de 2 nombres ou concaténation de 2 chaînes de caractères
-	Soustraction
*	Multiplication
/	Division
%	Modulo (Reste d'une division entière)
	Opérateur logique OU
&&	Opérateur logique ET
== et !=	Test d'égalité ou de différence (Attention en Java, différent de la méthode equals())

Expression conditionnelle

- Les expressions de type « *if-else* » peuvent être spécifiées en utilisant la syntaxe : **(*<condition>*) ? *<value on true>* : *<value on false>***

- Exemple :

```
(($F{name}.length() > 50) ? $F{name}.substring(0,50) :  
$F{name})
```

- Cette syntaxe est récursive comme dans :

```
(($F{name}.length() > 50) ?  
(($F{name}.startsWith("A")) ? "AAAA" : "BBB") :  
$F{name})
```

- Il est toujours possible de déléguer des expressions plus complexes à une méthode statique d'une classe Java

```
MyFormatter.toRomanNumber( $F{MyInteger}.intValue() )
```

L'éditeur d'expressions

- Ouvert par le bouton « ... » à côté de toute propriété acceptant une expression
- Colonne de gauche : paramètres, champs, variables, **fonctions intégrées** classées par catégorie (Texte, Date, Math, Logique)
- Double-clic pour insérer ; l'aide décrit chaque fonction avec un exemple
- Le résultat doit être du type attendu : texte pour un champ texte, vrai/faux pour Print When Expression
- Une erreur d'expression apparaît à la compilation, dans la vue Problems, avec le numéro de ligne

Concaténer des champs

- Opérateur + entre textes : `$F{prenom} + " " + $F{nom}`
- Ou la fonction `CONCATENATE($F{prenom}, " ", $F{nom})`
- Attention aux nombres : `$F{qte} + $F{prix}` additionne ; pour afficher, commencer par un texte : `" " + $F{qte} + " x " + $F{prix}`
- Un champ vide (null) s'affiche « null » : voir la gestion des valeurs nulles
- Retour à la ligne dans un champ texte : `$F{adresse} + "\n" + $F{cp} + " " + $F{ville}`
- Le champ texte doit être de classe `java.lang.String`

Afficher conditionnellement un texte

- Expression conditionnelle : `condition ? valeur_si_vrai : valeur_si_faux`
 - `$F{montant} > 1000 ? "Important" : "Standard"`
- Ou la fonction `IF($F{montant} > 1000, "Important", "Standard")`
- Masquer un élément : **Print When Expression**, ex. `$F{remise} > 0`
- Masquer une bande entière : Print When Expression de la bande
- Changer la couleur ou le gras selon une condition : style conditionnel
- Comparer des textes avec `EQUALS($F{statut}, "OK")`
ou `.equals(...)`, jamais avec `==`

Gérer une valeur vide ou nulle

- **Blank When Null** sur le champ texte : la valeur nulle devient une chaîne vide
- Valeur de remplacement : `$F{telephone} == null ? "Non renseigné" : $F{telephone}`
- Dans une concaténation : `(Objects.toString($F{prenom}, "") + " " + $F{nom}).trim()`
- Tester le vide et le nul : `$F{ref} == null || $F{ref}.trim().isEmpty()`
- Calculs : un nul dans une addition provoque une erreur → `($F{remise} == null ? 0 : $F{remise})`
- **Remove Line When Blank** : supprime l'espace occupé par un élément vide
- Préférer traiter les nuls dans la requête SQL (COALESCE) quand c'est possible

Une logique de formule Excel

- Les fonctions intégrées de JasperReports reprennent les noms Excel en anglais ; elles sont listées dans l'éditeur d'expressions

Formule Excel	Expression Jasper (fonction)	Équivalent Java
=SI(A1>100;"Oui";"Non")	IF(\$F{a} > 100, "Oui", "Non")	\$F{a} > 100 ? "Oui" : "Non"
=CONCATENER(A1;" ";B1)	CONCATENATE(\$F{a}, " ", \$F{b})	\$F{a} + " " + \$F{b}
=MAJUSCULE(A1)	UPPER(\$F{a})	\$F{a}.toUpperCase()
=GAUCHE(A1;3)	LEFT(\$F{a}, 3)	\$F{a}.substring(0, 3)
=ARRONDI(A1;2)	ROUND(\$F{a}, 2)	pattern #,##0.00
=AUJOURDHUI()	TODAY()	new java.util.Date()
=TEXTE(A1;"jj/mm/aaaa")	DATEFORMAT(\$F{d}, "dd/MM/yyyy")	pattern dd/MM/yyyy
=ESTVIDE(A1)	\$F{a} == null	\$F{a} == null
=SOMME(colonne)	Variable avec Calculation = Sum	\$V{total}

Calculs sur les dates

- Durée en jours entre deux dates : `DAYS($F{date_debut}, $F{date_fin})`
- En mois ou en années : `MONTHS(...)`, `YEARS(...)` ; jours ouvrés : `NETWORKDAYS(...)`
- Ajouter des jours : `DATE(YEAR($F{d}), MONTH($F{d}), DAY($F{d}) + 30)` ou `EDATE($F{d}, 1)` pour un mois
- Date du jour : `TODAY()` ; date et heure : `NOW()`
- Afficher : `DATEFORMAT($F{d}, "dd/MM/yyyy")` ou le pattern du champ texte
- Retard : `IF(DAYS($F{echeance}, TODAY()) > 0, "En retard", "À échoir")`
- Les champs doivent être de type date (`java.util.Date`, `java.sql.Date`, `java.sql.Timestamp`), pas texte

Construire des libellés plus parlants

- Un titre dynamique : "Commandes du client " + \$P{CLIENT} + " au " + DATEFORMAT(TODAY(), "dd/MM/yyyy")
- Traduire un code : IF(EQUALS(\$F{statut}, "V"), "Validée", IF(EQUALS(\$F{statut}, "A"), "Annulée", "En cours"))
- Quand il y a beaucoup de codes : faire la traduction dans la requête SQL (CASE WHEN) ou par jointure
- Pluriel : \$V{nb} + ((\$V{nb} > 1) ? " articles" : " article")
- Résumé de filtre en sous-titre : "Période : " + DATEFORMAT(\$P{debut}, "dd/MM/yyyy") + " - " + DATEFORMAT(\$P{fin}, "dd/MM/yyyy")
- Garder les expressions courtes ; au-delà de deux conditions imbriquées, passer par la requête ou par une variable

Évaluation des expressions

- Par défaut, les expressions sont évaluées immédiatement lorsque la moteur génère le rapport.
- On peut modifier ce comportement en modifiant l'attribut ***evaluationTime*** qui peut prendre les valeurs suivantes :
 - ***Now*** : Immédiatement (valeur par défaut)
 - ***Report*** : A la fin de la génération
 - ***Page*** : A la fin de la page
 - ***Column*** : A la fin de la colonne
 - ***Group*** : A la fin de la groupe
 - ***Band*** : A la fin de la bande dans laquelle se trouve l'expression
 - ***Auto*** : Lorsque l'expression dépend de variables interdépendantes, on laisse le choix au moteur pour choisir le bon moment pour l'évaluation

Atelier 9 : Affichage conditionnel, concaténation, durées

- Afficher « Nom Prénom » en gérant les prénoms absents
- Afficher une mention conditionnelle selon le montant et masquer un élément si la remise est nulle
- Calculer un nombre de jours entre deux dates et le qualifier (en retard / à échoir)
- Construire un titre dynamique à partir des paramètres du rapport

Ressources

- Documentation Jaspersoft Studio :
community.jaspersoft.com/documentation
- JasperReports Library, exemples et schéma :
jasperreports.sourceforge.net
- Code source et notes de version :
github.com/Jaspersoft/jasperreports
- « JasperReports Ultimate Guide » et « Jaspersoft Studio User Guide »
- Forums communautaires : community.jaspersoft.com, jasperforum.com

Merci de votre attention

David THIBAU